

TECHNICAL PROJECT REPORT

Celebrating My Favorite Human

A single-page birthday application

A seven-page, front-end-only React application built as a personal gift: an interactive birthday site combining a live countdown, a playable cricket scoreboard, a CSS-drawn cake, and a photo gallery — with every piece of content editable from a single data file.

Stack React 18 · Vite 8 · React Router 6 · Framer Motion

Type Client-side SPA, no backend

Deployment Vercel (static)

— Contents

1. Introduction and objectives
2. Technology stack and rationale
3. System architecture — routing, data layer, component model
4. Feature walkthrough
5. Design system — palette, typography, motion
6. Responsive and accessibility engineering
7. Testing and verification
8. Limitations and future work
9. Conclusion

1 Introduction and objectives

Celebrating My Favorite Human is a personal web application built as a birthday gift for a recipient living roughly 8,200 km from the author — Sri Lanka to Melbourne. The distance is the design brief: the site exists because the two people could not be in the same room, and several features address that directly rather than decoratively.

Objectives

1. **Emotional payload first.** The application is a gift, not a product. Interaction exists to carry sentiment — blowing out candles, flipping cards, hitting a six — rather than to complete a task.
2. **Editable without a developer.** All copy, dates, photos and counts live in one plain data file so the author can revise the content without touching component code.
3. **Zero infrastructure.** No backend, no database, no accounts. This is a privacy decision as much as a cost one: the recipient's birthday wish is stored only in his own browser and is unreadable by anyone, including the author.
4. **Phone-first.** The site will overwhelmingly be opened on a phone, from a message. It had to work there before anywhere else.
5. **Personal thematic coherence.** Cricket is the recipient's passion, so it runs through the site as a motif rather than sitting in an isolated section.

2 Technology stack and rationale

TECHNOLOGY	VERSION	ROLE AND RATIONALE
React	18.3	Component model. Chosen for the stateful interactions — candle state, flip cards, lightbox, the cricket game — which are awkward in plain DOM code.
Vite	8.1	Build tool and dev server. Near-instant HMR matters here because the workflow is heavily iterative on copy and CSS.
React Router	6.28	Client-side routing across the seven pages, with a catch-all 404.
Framer Motion	11.11	Scroll-triggered reveals and presence animations. Declarative <code>whileInView</code> avoids hand-written IntersectionObserver code.
canvas-confetti	1.9	The single celebratory burst when the last candle goes out.

Why no backend

A server was deliberately excluded. The only user-generated content is the birthday wish, and the most meaningful property that wish can have is that *nobody else can read it*. Persisting it to `localStorage` makes that guarantee structural rather than a promise. It also reduces the deployment to a static bundle, which can be hosted free and indefinitely.

3 System architecture

3.1 Project structure

```
src/
├─ main.jsx           entry point, Router provider
├─ App.jsx            route table + persistent chrome
├─ index.css          design tokens + shared primitives
├─ data/
│   ├─ site.js        ALL content – 311 lines, single source of truth
│   └─ nav.js         page list, read by both navbar and footer
├─ components/        9 reusable pieces
│   ├─ Navbar · Footer · PageHeader · ScrollToTop
│   ├─ Countdown      self-correcting date maths
│   ├─ NotOut         the cricket scoreboard + bat-and-ball game
│   ├─ CreaseScroll    scroll progress as a rolling cricket ball
│   └─ FloatingHearts · Sparkles    ambient decoration
└─ pages/             8 route components (7 pages + 404)
```

3.2 The data layer

The defining architectural decision is that no component owns copy. Every string, date, photo path and count is exported from `src/data/site.js` and imported where needed. Thirteen named exports partition the content by page or feature:

EXPORT	GOVERNS
<code>him</code> / <code>her</code>	Names, nicknames, birthday, cities
<code>distance</code>	Kilometres apart, timezone gap, next meeting date
<code>images</code>	All photo paths
<code>home</code> , <code>notOut</code>	Hero copy, stat cards, scoreboard, shirt number
<code>story</code> , <code>reasons</code>	Timeline chapters, flip-card pairs
<code>longDistance</code> , <code>letter</code>	Prose blocks and the letter body
<code>cake</code> , <code>gallery</code> , <code>footer</code>	Candle count, wish prompts, photo captions

Collections are rendered by mapping, so the site adapts to their length automatically: adding a thirteenth entry to `reasons` produces a thirteenth card with no code change, and `cake.candles` controls both how many candles are drawn and the win condition for the page.

3.3 Application shell

`App.jsx` holds the route table and four persistent elements that sit outside it: the navbar, the footer, the scroll-progress indicator, and a `ScrollToTop` component that resets scroll position on navigation — a necessary

correction, since client-side routing otherwise preserves the previous page's scroll offset.

3.4 State management

No global state library is used, and none is warranted: no interactive state is shared across routes. Each page owns its own state locally via `useState` — lit candles and the saved wish on the cake page, the active photo index in the gallery, flipped cards on the reasons page, runs and sixes in the cricket game. The only cross-session persistence is the wish in `localStorage`.

One deliberate exception to idiomatic React: the scroll-progress component writes transforms directly to the DOM through a ref rather than through state, because it fires on every scroll frame and re-rendering the tree at that rate would be wasteful.

4 Feature walkthrough

ROUTE	PAGE	PRINCIPAL FEATURES
/	Home	Hero with polaroid; live countdown; three stat figures; the "27 Not Out" cricket scoreboard with playable bat-and-ball; navigation tiles
/cake	Your Cake	CSS-drawn cricket cake; individually extinguishable candles; room dims and confetti fires on the last one; wish saved to <code>localStorage</code>
/our-story	Our Story	Chapter timeline, zig-zagging left/right above 860px and collapsing to a single column below
/reasons	Reasons	3D flip cards on a responsive grid, with a flip-all control and progress counter
/gallery	Gallery	CSS-column masonry; full-screen lightbox with keyboard navigation, focus handling and body-scroll locking
/long-distance	The Distance	Two-city map band with animated flight path; distance and timezone figures; countdown to the next meeting
/letter	Letter	The birthday letter typeset on ruled paper with a postage stamp and an inline floated photograph

4.1 The countdown

The countdown ticks once per second toward the next birthday and, once the date passes, rolls over to the following year automatically by comparing against the current date rather than a stored target. On the day itself it swaps to a celebratory state. The component therefore never requires the date to be edited again.

4.2 The cricket motif

Rather than confining cricket to one page, it is distributed through three touchpoints:

- **The scoreboard** on the home page presents the recipient's age as a batting score, days alive as balls faced, and his shirt number on a CSS-drawn batter. Tapping the ball plays a shot; six sixes completes a perfect over.
- **The cake** is a cricket cake — a mown outfield with a pitch and creases on the top tier, the score piped on the middle tier, a leather ball on its shoulder, and a seam-stitched band at the base.
- **Scroll progress** is shown as a cricket ball rolling along a crease at the top of the viewport, its rotation derived from distance travelled so that it genuinely rolls rather than slides.

4.3 Illustration technique

The cake, the batter, the stumps, the bat and both balls are drawn entirely in CSS — gradients, border-radius, clip-path and masks — with no image assets and no icon font. Each tier of the cake, for example, is composed of an elliptical top surface plus a body shaded across its width so it reads as a cylinder rather than a rectangle. This keeps the payload small, allows the illustrations to inherit the palette, and lets them scale losslessly.

5 Design system

5.1 Colour

All colour is declared as custom properties at the top of `index.css`. A warm romantic base — rose, coral, peach, gold — is widened with lilac, mint and sky so the palette reads joyful rather than uniformly warm.

Six accent hues are exposed as a rotating set through utility classes `tone-1` ... `tone-6`. Each tone declares four values — a flat hue, a gradient, a darker gradient safe behind white text, and a faint wash — and components consume those variables rather than hard-coding colour. Applying `tone-4` to a card recolours its rule, icon, hover shadow and wash together, and the whole rotating set can be retuned from one place.

5.2 Typography

FACE	ROLE
Fraunces (serif)	Display headings and numerals; carries the editorial tone
Outfit (sans)	Body copy, UI labels, letterspaced small caps
Caveat (script)	The personal voice — captions, asides, signatures

Type scales fluidly via `clamp()` rather than stepping at breakpoints, so headings resize continuously between the smallest phone and a desktop monitor.

5.3 Motion

Motion is used to reward interaction rather than to decorate load. Content reveals on scroll with a short upward fade; interactive elements respond on hover or press; and celebratory moments — the six, the last candle — are the only places with large-amplitude animation. All of it is suppressed under `prefers-reduced-motion`.

6 Responsive and accessibility engineering

The site is built mobile-first: single-column layouts are the default and `min-width` queries progressively introduce multi-column arrangements. Five problems required specific engineering beyond that.

6.1 Hover states on touchscreens

Every decorative hover effect — lift, scale, colour wash — was originally unguarded. On a touchscreen the hover state *latches* after a tap and persists, leaving elements stuck mid-animation. All such rules are now wrapped in `@media (hover: hover)`, with `:active` press states provided for touch instead.

Two cases were more serious than cosmetic, because hover was concealing *content*: gallery captions were fully transparent until hover, so on a phone they could never be read at all; and the "tap to flip" cue on the reasons cards — the only affordance signalling that the cards do anything — was likewise hidden. Both are now shown outright on touch devices via `@media (hover: none)`.

6.2 Viewport units

Full-height sections used `100vh`, which on mobile browsers resolves to the viewport *without* the address bar and therefore overflows the visible area. These use `100svh` — the small viewport height — so a full-height hero can never push its own call to action off screen.

6.3 Safe-area insets

With `viewport-fit=cover` the layout extends under the notch and home indicator. A `--nav-total` token composes the navbar height with `env(safe-area-inset-top)`, and every element that keys off the navbar — page padding, the scroll indicator, the mobile drawer — derives from that single token.

6.4 The lightbox

Laid out as a flex row, the gallery lightbox's two 48px navigation arrows consumed roughly 110px of a 320px screen, leaving the photograph barely half the width. The arrows were lifted out of flow to overlay the image edges. Because the photographs are tall — roughly 2:3.5 — height rather than width was the binding constraint, so the figure is capped to the available height and flex distributes whatever the caption does not use. Photo width rose from 77% to 93% of the viewport.

6.5 Touch targets

All interactive elements meet the 44px minimum. The cake's candles are a constrained case: eight candles on a ~230px cake cannot each be 44px wide, so each candle button instead claims an equal flex share of the row, with the drawn candle centred inside it — yielding targets of 22–30px wide by 96px tall that can never overlap regardless of candle count.

7 Testing and verification

Verification was performed by driving a headless Chromium instance against a production build and measuring the rendered result, rather than by visual inspection alone. Emulation used a mobile user-agent and touch emulation so that `hover` and `pointer` media features resolved as they do on a real device.

7.1 Coverage and results

CHECK	METHOD	RESULT
Horizontal overflow	7 pages × 7 widths (320–1440px); compared <code>scrollWidth</code> against viewport	49 / 49 PASS
Touch target size	Measured every <code><a></code> and <code><button></code> against the 44px minimum	0 UNDERSIZED
Reduced motion	Emulated <code>prefers-reduced-motion: reduce</code> ; counted running animations	0 ANIMATING
Touch media queries	Asserted <code>(hover: none)</code> matches under touch emulation	PASS
Gallery captions	Computed opacity of caption overlay without hover	1.0 VISIBLE
Lightbox sizing	Measured photo width as a fraction of viewport at 320 and 390px	93% / 94%
Landscape drawer	Opened menu at 740×360; confirmed all 7 links reachable	PASS
Cake geometry	Asserted no candle falls outside the top tier, 5 widths	0 OFF-TIER

7.2 Defects found by measurement

Several defects were invisible to reading the source and were caught only because the rendered output was measured:

- **A dead media query.** A rule reducing drawer type size in landscape was placed *above* the base rule it intended to override. With equal specificity, source order won and the override never applied. Landscape now shows 6 of 7 links at once rather than 4.
- **An overflowing label.** The countdown's `SECONDS` label, a single unbreakable word, measured wider than its ~57px tile at 320px and spilled out.
- **An undersized control.** The menu button measured 42×42px — just under the 44px minimum, and the only route to navigation on a phone.
- **A stacking-context trap.** A cricket ball placed inside a cake tier was invisible: the tier established its own stacking context, so the ball's `z-index` could not lift it above a sibling tier painted on top.

7.3 Build output

ASSET	RAW	GZIPPED
JavaScript bundle	334.2 kB	108.4 kB
Stylesheet	59.7 kB	13.2 kB

ASSET	RAW	GZIPPED
HTML document	1.2 kB	0.6 kB
Total transferred	—	≈122 kB

Excludes photography and web fonts, which are fetched separately. Source comprises roughly 1,900 lines of JSX and 4,400 of CSS.

8 Limitations and future work

8.1 Known limitations

- **Bundle size.** At ~108 kB gzipped of JavaScript, Framer Motion is a substantial share of the payload for what is largely scroll-reveal animation. Much of it could be replaced with CSS scroll-driven animations or a small IntersectionObserver hook.
- **No image optimisation pipeline.** Photographs are served as authored. Responsive `srcset` and modern formats such as AVIF would materially reduce transfer on phones.
- **Fixed illustration geometry.** Elements of the cake are positioned with pixel offsets measured from its base. Changing a tier height requires those offsets to be adjusted by hand.
- **Single-locale.** Copy is English-only and dates are formatted for one locale.
- **No automated regression suite.** Verification was performed with purpose-written scripts rather than a committed test suite, so the checks are reproducible but not enforced on change.

8.2 Future work

- Commit the verification scripts as a Playwright suite so the overflow, tap-target and reduced-motion checks run automatically.
- Add swipe gestures to the gallery lightbox — currently navigable by arrows and keyboard, but swiping is the expected phone interaction.
- Introduce an image pipeline generating responsive variants at build time.
- Audit colour contrast programmatically against WCAG AA, and add a skip-to-content link.
- Drive the cake illustration's geometry from custom properties so tier dimensions and dependent offsets stay in sync automatically.

9 Conclusion

The application meets its objectives. It is a seven-page interactive site with no backend, deployable as a static bundle, in which every piece of content is editable from a single 311-line data file by someone who does not write code.

The most instructive part of the work was the responsive and accessibility pass. The layout was already mobile-first and looked correct in a resized desktop browser; the defects that mattered — latched hover states hiding captions, a dead media query, a control two pixels under the touch minimum, a viewport unit that overflows only when a real address bar is present — were invisible to inspection and surfaced only by emulating a touch device and measuring what was actually rendered. That distinction, between a layout that reflows and an interface that genuinely works under a thumb, is the substantive engineering result of the project.

Technically the site is unremarkable: no backend, no state library, no novel algorithms. Its interest lies in constraint. Restricting all content to one data file forced a clean separation between content and presentation; refusing a server made the privacy of the recipient's wish a structural property rather than a policy; and drawing every illustration in CSS kept the payload under 122 kB while letting the artwork inherit the palette and scale losslessly.